

SESSION 8

# การทดสอบและการ Deploy

Vitest · React Testing Library · MSW · Vite Build · Vercel

---

หลักสูตร React-Redux · โปรเจกต์ AcadeMate

Dr.Watcharin Sarachai

## Vitest + RTL

ทดสอบ Unit & Component

## MSW

จำลอง API ระดับ Network

## Async Testing

Optimistic Update & Rollback

## Deploy

Vite Build + Vercel CI

## บทที่ 8 — แผนการเรียนรู้และวัตถุประสงค์การเรียนรู้

### Part 1 — Testing Setup

ตั้งค่า Vitest + jsdom สร้าง renderWithProviders และติดตั้ง MSW

### Part 2 — Unit Testing

ทดสอบ reducers, selectors และ React components ด้วย Testing Library

### Part 3 — MSW API Mocks

ดักจับ network requests ของ RTK Query และจำลอง response ทั้งสำเร็จและข้อผิดพลาด

### Part 4 — Testing Async Patterns

ตรวจสอบว่า optimistic updates ปรับปรุง cache และ rollback เมื่อ server ล้มเหลว

### Part 5 — Build & Deploy

Vite production build, code splitting, environment variables และ deploy ไปยัง Vercel

### Lab 8 — AcadeMate Test Suite

เขียน integration tests สำหรับ StudentTable และ AddStudentForm แล้ว deploy ไปยัง Vercel

## SESSION 8

# การติดตั้งระบบทดสอบ

---

ตั้งค่า Vitest ร่วมกับ jsdom, Testing Library และ MSW ในโปรเจกต์ AcadeMate

# ภาพรวมชุดเครื่องมือทดสอบ

เครื่องมือสามชนิดครอบคลุมทุกชั้นของแอป AcadeMate

| Vitest                                                                                                            | ตัวประมวลผลการทดสอบ | React Testing Library                                                                                                    | ทดสอบ Component | MSW (Mock Service Worker)                                                                                               | จำลองชั้น API |
|-------------------------------------------------------------------------------------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------|---------------|
| API แบบ Vite-native เข้ากันได้กับ Jest<br>รันใน jsdom รองรับ<br>describe/it/expect, coverage และ --<br>watch mode |                     | Render components ลงใน DOM จริง<br>ค้นหาด้วย role, label และข้อความ เน้น<br>ทดสอบพฤติกรรมผู้ใช้ ไม่ใช่<br>implementation |                 | ดักจับ network requests ระดับ<br>fetch/XHR ไม่ต้องแก้โค้ด RTK Query รับ<br>responses จริงโดยไม่ต้องเรียก server<br>จริง |               |

แต่ละเครื่องมือทดสอบอะไร

Reducers & Selectors

React Components

การผสาน RTK Query

Vitest

RTL

Vitest+MSW

Pure ฟังก์ชัน — ไม่ต้องใช้ DOM หรือ network

Render → ได้ตอบ → ตรวจสอบข้อความ/roles

Dispatch query → MSW ตอบสนอง → ตรวจสอบ cache

# ติดตั้ง Dependencies สำหรับทดสอบ & vite.config.js

เพิ่ม Vitest, RTL และ MSW เป็น dev dependencies จากนั้นตั้งค่า test environment ใน vite.config.js

```
# Install all testing dependencies
npm install -D vitest @vitest/ui jsdom
npm install -D @testing-library/react @testing-library/jest-dom
npm install -D @testing-library/user-event
npm install -D msw

// vite.config.js - add test block
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',    // browser-like DOM in Node
    globals: true,          // use describe/it/expect without imports
    setupFiles: ['./src/tests/setup.js'],
    coverage: {
      reporter: ['text', 'html'],
      include: ['src/**/*.{js,jsx}'],
    },
  },
});
```

# Vitest คืออะไร?

เฟรมเวิร์กสำหรับทดสอบ Unit Test ที่ออกแบบมาสำหรับ Vite

## Vitest คืออะไร?

- เฟรมเวิร์กทดสอบ JavaScript รุ่นใหม่ทำงานบน Vite
- API เข้ากันได้กับ Jest ไม่ต้องเรียนรู้ใหม่ทั้งหมด
- รองรับ JavaScript, ESM และ JSX แบบ native
- รันเร็วกว่า Jest มากเพราะใช้ Vite pipeline
- รองรับ --watch mode และ coverage แบบ built-in

## จุดเด่น

**Fast** รันเทสต์เร็วขึ้น 2-5x

**Smart** รู้ว่าไฟล์ไหนเปลี่ยน

**Simple** ตั้งค่าน้อย ใช้งานง่าย

**Compatible** ใช้แทน Jest ได้เลย

# Vitest ทำงานอย่างไร?

กระบวนการทำงานของ Vitest ตั้งแต่เขียนโค้ดถึงรายงานผล



## คำสั่งสำคัญ

**`npx vitest`**

รันเทสได้ใน watch mode

**`npx vitest run`**

รันครั้งเดียวแล้วออก

**`npx vitest --coverage`**

รันพร้อมรายงาน coverage

**`npx vitest ui`**

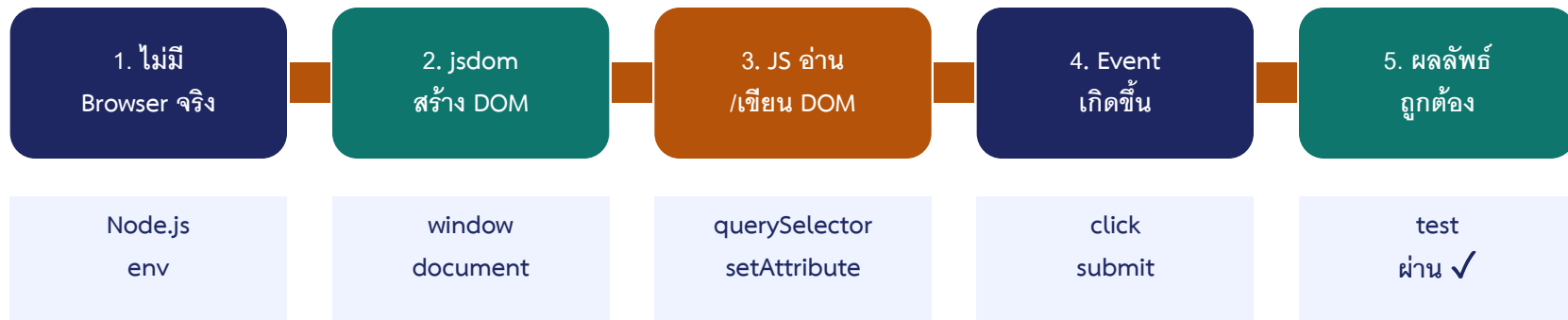
เปิด UI แสดงผลแบบ visual



**Tip:** Vitest ใช้ vite.config.js เดิมของโปรเจกต์ — ไม่ต้องตั้งค่าแยก  
เพียงเพิ่ม **`test: { environment: 'jsdom' }`** ใน vite.config.js

# jsdom คืออะไร?

จำลอง Browser Environment ใน Node.js สำหรับการทดสอบ



## ฟีเจอร์หลักของ jsdom

|                                        |                                 |                                    |                              |
|----------------------------------------|---------------------------------|------------------------------------|------------------------------|
| DOM API<br>document / window / element | CSS Support<br>getComputedStyle | Events<br>addEventListener / click | Forms<br>submit / validation |
|----------------------------------------|---------------------------------|------------------------------------|------------------------------|

💡 **Tip:** jsdom ไม่ใช่ Browser จริง — ไม่รองรับ CSS animations, WebGL, หรือ layout จริง ใช้เพื่อทดสอบ logic DOM เท่านั้น

ตั้งค่าใน Vitest: `test: { environment: 'jsdom' }`



# ตัวอย่างการใช้งาน Vitest

## เขียนและรัน Unit Test ด้วย Vitest ใน JavaScript

### math.js (source file)

```
// math.js
export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}
```

### math.test.js (test file)

```
import { describe, it, expect } from 'vitest'
import { add, multiply } from './math.js'

describe('Math functions', () => {
  it('adds two numbers', () => {
    expect(add(2, 3)).toBe(5)
  })
  it('multiplies two numbers', () => {
    expect(multiply(4, 5)).toBe(20)
  })
})
```



### อธิบายโค้ด

**describe()** — จัดกลุ่ม test ที่เกี่ยวข้องกัน

**it() / test()** — นิยาม test case แต่ละอัน

**expect()** — ระบุค่าที่ต้องการตรวจสอบ

**.toBe()** — ตรวจสอบว่าค่าเท่ากันแบบ strict

**.toEqual()** — ตรวจสอบ object ว่า deep equal

**.toBeTruthy()** — ตรวจสอบว่าค่าเป็น truthy

```
✓ adds two numbers (2ms)
✓ multiplies two numbers (1ms)
Tests 2 passed | 0 failed
```

# Vitest API: describe / it / expect

## โครงสร้างพื้นฐานของ test suite ใน Vitest

math.test.js

```
// math.test.js
import { add } from './math'

describe('Math', () => {
  it('adds numbers', () => {
    expect(add(2, 3)).toBe(5)
  })
  it('multiplies', () => {
    expect(mul(3, 4)).toBe(12)
  })
})
```

### describe(name, fn)

- ความหมาย: จัดกลุ่ม test ที่เกี่ยวข้องกัน
- รูปแบบ: `describe('Math', () => { ... })`  
💡 ซ่อน describe ได้หลายชั้น

### it(name, fn) / test(name, fn)

- ความหมาย: นิยาม test case แต่ละอัน
- รูปแบบ: `it('adds', () => { ... })`  
💡 it และ test ใช้แทนกันได้ทุกที่

### expect(value)

- ความหมาย: ระบุค่าที่ต้องการตรวจสอบ
- รูปแบบ: `expect(add(2, 3))` → ต่อด้วย **matcher**  
💡 ใช้คู่กับ matcher เสมอ เช่น `.toBe()`

💡 Tip: describe ซ่อน describe ได้ — ใช้จัดโครงสร้างตาม feature / scenario / unit ตามที่ต้องการ

# Vitest Matchers: .toBe / .toEqual / .toBeTruthy

Matcher — ฟังก์ชันที่บอก Vitest ว่าต้องการตรวจสอบอะไร

## ตัวอย่าง matchers

```
// .toBe - strict equal (===)
expect(2 + 3).toBe(5) // ✓
expect({a:1}).toBe({a:1}) // ✗ ไม่ผ่าน!

// .toEqual - deep comparison
expect({a:1}).toEqual({a:1}) // ✓ ผ่าน
```

## .toBe(expected)

- ตรวจสอบว่าค่าเหมือนกันแบบ strict (===)

ใช้กับ primitive: number, string, boolean

ไม่ใช้กับ object หรือ array

- ตัวอย่าง:

```
expect(5).toBe(5) ✓
expect({}).toBe({}) ✗
```

## .toEqual(expected)

- ตรวจสอบ object/array ว่า deep equal

เปรียบเทียบทุก property แบบ recursive

ค่าเหมือนกัน แม้คนละ reference ก็ผ่าน

- ตัวอย่าง:

```
expect({a:1}).toEqual({a:1}) ✓
```



จำง่าย: **.toBe** = ค่าตรงๆ | **.toEqual** = object/array

# Vitest Matchers: .toBe / .toEqual / .toBeTruthy

Matcher — ฟังก์ชันที่บอก Vitest ว่าต้องการตรวจสอบอะไร

## ตัวอย่าง matchers

```
// .toBeTruthy - truthy check
expect('hello').toBeTruthy() // ✓
expect(0).toBeTruthy()      // ✗
```

## .toBeTruthy()

- ตรวจสอบค่าเป็น truthy

falsy: false, 0, "", null, undefined, NaN

truthy: ค่าอื่นทั้งหมด

- ตัวอย่าง:

```
expect('ok').toBeTruthy() ✓
expect(0).toBeTruthy()   ✗
```



จำง่าย: **.toBe** = ค่าตรงๆ | **.toEqual** = object/array | **.toBeTruthy** = มีค่า

# Vitest vs Jest เปรียบเทียบ

เลือกใช้เครื่องมือที่เหมาะสมกับโปรเจกต์ของคุณ

| หัวข้อ        |  Vitest |  Jest | แนะนำ   |
|---------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|---------|
| ความเร็ว      | เร็วมาก (Vite pipeline)                                                                  | ปานกลาง (Babel)                                                                          | Vitest  |
| JavaScript    | รองรับ native ไม่ต้อง config                                                             | รองรับ native ใช้งานได้ทันที                                                             | Vitest  |
| ESM Support   | รองรับ native                                                                            | ต้องตั้งค่าเพิ่ม                                                                         | Vitest  |
| Config ไฟล์   | ใช้ vite.config.ts                                                                       | jest.config.js แยก                                                                       | Vitest  |
| Ecosystem     | ใหม่ กำลังเติบโต                                                                         | สมบูรณ์ มาก plugin                                                                       | Jest    |
| CI/CD         | รองรับ --run mode                                                                        | รองรับดี นิยมมาก                                                                         | ทั้งคู่ |
| React Testing | ใช้กับ RTL ได้                                                                           | ใช้กับ RTL ได้                                                                           | ทั้งคู่ |

✅ สรุป: โปรเจกต์ใหม่ที่ใช้ Vite แนะนำ Vitest | โปรเจกต์เดิมที่มี Jest อยู่แล้วก็ยังสามารถใช้ Jest ได้เลย

# render() ใน App จริง vs ใน Test

ความแตกต่างของ environment ที่ทำให้ต้องเตรียม context ให้ครบก่อนทดสอบ

## 🌐 App จริง (Browser)

- มี **main.jsx**
- มี **Redux Store**
- มี Router / Provider ห่อไว้



## 🧪 Test Environment (jsdom)

- ไม่มี **main.jsx** — เริ่มจากเปล่า
- ไม่มี **Redux Store** โดย default
- ไม่มี Router / Provider ห่อไว้
- ต้องเตรียม context ด้วยตัวเอง

```
// main.jsx — context ครบ ไม่ต้องทำอะไรเพิ่ม
createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <App /> </Provider>
)
```

```
// ❌ render() ขรรมดา — ไม่มี store → พัง
render(<StudentTable />)

// Error: could not find react-redux
// context value
```

✅ แนวทาง: เตรียม context ให้ครบก่อน render — ใช้ **renderWithProviders()** ห่อ store ให้อัตโนมัติ

# renderWithProviders

## ❌ ปัญหา: render() ธรรมดาไม่พอ

เมื่อ component ใช้ RTK Query hooks เช่น `useGetStudentsQuery()` จะเกิดข้อผิดพลาดทันที หากไม่มี Redux store:

```
// ❌ พังทันที — ไม่มี store
render(<StudentTable />);
```

Error: could not find react-redux context

RTL ไม่รู้จัก Redux — ต้องห่อ `<Provider>` ไว้ข้างนอกก่อนเสมอ

## ✅ แนวทางแก้ไข

สร้างฟังก์ชัน `renderWithProviders()` ที่:

- สร้าง Redux store ใหม่ทุกครั้งที่เราเรียก
- ห่อ component ด้วย `<Provider store={store}>` อัตโนมัติ
- แต่ละ test ได้ store แยกกัน — ไม่กระทบกัน
- `preloadedState` เพิ่มข้อมูลเริ่มต้นให้ store ได้
- คืน `{ store, ...renderResult }` — ใช้งานเหมือน `render()` ทั่วไป

```
// ✅ ใช้งานแทนได้ทันที
renderWithProviders(<StudentTable />);
```

# renderWithProviders — ภาพรวม 3 ขั้นตอน

ฟังก์ชันทำงาน 3 อย่างต่อกัน — แต่ละหน้าถัดไปจะเจาะลึกทีละขั้น

## ขั้นที่ 1: สร้าง Store ใหม่

- `configureStore()` ทุกครั้ง
- ใส่ `reducer + middleware`
- รับ `preloadedState`



## ขั้นที่ 2: สร้าง Wrapper

- React component พิเศษ
- ห่อ children ด้วย `<Provider>`
- ส่ง store เข้า Provider



## ขั้นที่ 3: คำนวณค่า + Render

- เรียก `RTL render()` จริง
- ส่ง Wrapper เป็น option
- คืน `{ store, ...queries }`



ได้รวมทั้งฟังก์ชัน

```
export function renderWithProviders(ui, { preloadedState = {} } = {}) {  
  const store = configureStore({...}); // ขั้น 1  
  function Wrapper({children}) { return <Provider store={store}>{children}</Provider>; } // ขั้น 2  
  return { store, ...render(ui, { wrapper: Wrapper }) }; // ขั้น 3  
}
```



# renderWithProviders — ขั้นตอนที 1

## 🛠️ ขั้นตอนที่ 1: สร้าง Redux Store ใหม่

```
// รับ 2 parameter:  
// ui = component ที่จะ render  
// preloadedState = ข้อมูลเริ่มต้น  
export function renderWithProviders(  
  ui,  
  { preloadedState = {} } = {}  
) {  
  // ← สร้าง store ใหม่ทุกครั้ง  
  const store = configureStore({  
    reducer: {  
      [studentsApi.reducerPath]: studentsApi.reducer,  
    },  
    middleware: (gDM) =>  
      gDM().concat(studentsApi.middleware),  
    preloadedState,  
  });  
  // ขั้นตอนที่ 2 และ 3 ต่อไป...
```

### ทำไมต้องสร้างใหม่ทุกครั้ง?

ถ้าใช้ store เดิมทำการ test — state จากการ test ก่อนหน้าจะ "รั่ว" เข้ามา ทำให้ผลลัพธ์ไม่น่าเชื่อถือหรือไม่ถูกต้อง

### reducer คืออะไร?

บอกว่า store มี "slice" อะไรบ้าง — ที่นี้มีแค่ studentsApi เป็น slice เดียว (slice ทำงานกับ store จำลองที่สร้างขึ้นมาใหม่)

### middleware คืออะไร?

RTK Query ต้องการ middleware พิเศษเพื่อจัดการ cache, polling, และ subscription — ถ้าขาดจะ error

### preloadedState คือ?

ข้อมูลเริ่มต้นที่ยัดเข้า store ก่อนรัน test — ใช้จำลองสถานะต่างๆ เช่น "มีข้อมูลแล้ว" หรือ "กำลังโหลด"

# renderWithProviders — ขั้นตอนที่ 2



## ขั้นตอนที่ 2: สร้าง Wrapper Component

```
// ...ต่อจากขั้นตอนที่ 1...  
// Wrapper = React component พิเศษ  
// ที่ห่อ children ด้วย <Provider>  
function Wrapper({ children }) {  
  return (  
    <Provider  
      store={store}  
    >  
      {children}  
    </Provider>  
  );  
}
```

### children คืออะไร?

component ที่ส่งเข้ามาให้ render เช่น <StudentTable /> — Wrapper จะห่อให้อัดโน้มนิด

### ทำไมต้องเป็น function?

RTL ต้องการ React Component ที่เรียกซ้ำได้ จึงต้องเป็น function ไม่ใช่ JSX element

### store={store} เชื่อมที่ไหน?

store ที่ส่งเข้า Provider คือ store ที่สร้างในขั้นตอนที่ 1 — ทั้งคู่อยู่ใน closure เดียวกัน

### เกิดอะไรขึ้นเมื่อ render?

RTL จะเรียก Wrapper({children: <StudentTable/>}) ก่อน แล้ว StudentTable จะเห็น store ผ่าน Context

# renderWithProviders — ขั้นตอนที่ 3



## ขั้นตอนที่ 3: เรียก render() และคืนค่า

```
// ...ต่อจากขั้นตอนที่ 2...
// ส่ง ui + Wrapper เข้า RTL render()
// รวม store กลับด้วย spread operator
return {
  store, ...render(ui, {
    wrapper: Wrapper
  }),
};
}
```



## ค่าที่ได้รับกลับมา

**store** — Redux store — ใช้ dispatch() / getState() ใน test ได้

**container** — DOM node หลักที่ render อยู่

**getByText, getByRole...** — RTL queries ทั้งหมด ใช้ค้นหา element

**debug()** — print DOM ออกมาดู ใช้ debug ได้เลย

## ...render() คืออะไร?

RTL render() ธรรมดา — แต่ส่ง Wrapper เป็น wrapper option ทำให้ component ถูกห่อด้วย Provider ก่อน mount

## spread ... ทำงานอย่างไร?

{ store, ...render(...) } รวม store เข้ากับ object ที่ render() คืนมา ผู้เรียกได้ทั้ง store และ RTL queries จากที่เดียว

## ใช้งานอย่างไรใน test?

const { store, getByText } = renderWithProviders(<StudentTable />)  
แล้วใช้ getByText หา element หรือ store.dispatch() ได้เลย

## wrapper: Wrapper คืออะไร?

option ของ RTL render() — บอกให้ RTL ห่อ ui ด้วย Wrapper ก่อน render จริง เหมือนที่ main.jsx ทำด้วย <Provider>

# การใช้งาน renderWithProviders

## ตัวอย่างการเรียกใช้และข้อควรระวัง

### ✓ กรณีที่ 1 — Basic usage

```
import { renderWithProviders } from
'../tests/utils'

renderWithProviders(<StudentTable />)
```

ใช้ store เปล่า — เหมาะกับ component ที่ไม่ต้องการ state เริ่มต้น

### ✓ กรณีที่ 2 — Preloaded state

```
renderWithProviders(
  <StudentTable />,
  {
    preloadedState: {
      students: { list: mockData, status: 'succeeded' }
    }
  }
)
```

ส่ง preloadedState เพื่อกำหนด state เริ่มต้นให้ test

### ⚠ ข้อควรระวัง

#### Fresh store ทุก test

แต่ละ test ได้ store ใหม่เสมอ — ไม่มี state หลงเหลือระหว่าง test

#### Middleware ทำงานปกติ

thunk และ middleware อื่นๆ ใช้งานได้ใน renderWithProviders เหมือน production

#### cleanup อัตโนมัติ

React Testing Library เคลียร์ DOM หลังแต่ละ test โดยอัตโนมัติ

# renderWithProviders — โค้ดแบบเต็ม

src/tests/utlis.jsx — wrapper ครอบ Redux store สำหรับทดสอบ

src/tests/utlis.jsx

```
import { render } from '@testing-library/react'
import { Provider } from 'react-redux'
import { configureStore } from '@reduxjs/toolkit'
import { studentsApi } from '../features/students/studentsApi'

export function renderWithProviders(ui, {
  preloadedState = {}
} = {}) {
  const store = configureStore({
    reducer: {
      [studentsApi.reducerPath]: studentsApi.reducer
    },
    middleware: gDM => gDM().concat(studentsApi.middleware),
    preloadedState
  })
  function Wrapper({ children }) {
    return <Provider store={store}>{children}</Provider>
  }
  return { store, ...render(ui, { wrapper: Wrapper }) }
}
```

## อธิบายโค้ด

### configureStore

สร้าง Redux store ใหม่ทุกครั้ง ไม่ใช่ store หลัก

### preloadedState

ส่ง initial state เพื่อเซตข้อมูลก่อน render

### studentsApi.reducer

ลงทะเบียน RTK Query reducer และ middleware

### Wrapper

ครอบ component ด้วย Provider ก่อน render

### return { store, ...render }

คืน store + queries ทุกตัวจาก RTL



renderWithProviders ใช้แทน render() ของ RTL ในทุก test ที่ต้องการ Redux store

SESSION 8 — Unit Testing

# Unit Testing

ทดสอบ Reducers, Selectors และ React Components

---

Vitest · RTL · renderWithProviders · RTK Query

แยกส่วนการทดสอบแต่ละชั้นของแอปพลิเคชัน

# การทดสอบ Slice Reducer

Reducers เป็น pure function — ส่ง action โดยตรง ไม่ต้องตั้งค่า store

## studentsSlice.test.js

```
import studentsReducer from './studentsSlice'
import { describe, it, expect } from 'vitest'

describe('studentsSlice', () => {
  const initialState = studentsReducer(
    undefined, { type: '@@INIT' }
  )

  it('starts with status: idle', () => {
    expect(initialState.status).toBe('idle')
  })

  it('sets loading on fetchStudents.pending', () => {
    const action = { type: 'students/fetchAll/pending' }
    const state = studentsReducer(initialState, action)
    expect(state.status).toBe('loading')
  })

  it('populates entities on fulfilled', () => {
    const students = [{ id: '1', name: 'Alice', gpa: 3.9 }]
    const action = {
      type: 'students/fetchAll/fulfilled',
      payload: students
    }
    const state = studentsReducer(initialState, action)
    expect(state.status).toBe('succeeded')
    expect(state.list).toEqual(students)
  })
})
```

## หลักการทดสอบ Reducer

### Pure Function

Reducer คำนวณ state ใหม่เสมอ — ไม่มี side effect

### @@INIT action

ส่ง action นี้เพื่อตั้ง initialState โดยไม่มี store

### pending / fulfilled

ทดสอบแต่ละ action lifecycle แยกกัน

### expect().toBe()

ตรวจสอบ status string ให้ตรงกับ state machine

### expect().toEqual()

ตรวจสอบ array/object ให้ deep equal กับ payload

# การทดสอบ React Components ด้วย RTL

ใช้ `renderWithProviders` แทน `render()` เพื่อให้ Redux hooks ทำงานได้ใน test

## StudentTable.test.jsx

```
// @vitest-environment jsdom
import '@testing-library/jest-dom/vitest';
import { screen, waitFor } from '@testing-library/react';
import { describe, expect, it } from 'vitest';
import { renderWithProviders } from '../tests/utils';
import StudentTable from './StudentTable';
const mockStudents = [
  { id: 1, name: "Alice", studentId: "S001", major: "CS", gpa: 3.8 },
  { id: 2, name: "Bob", studentId: "S002", major: "Math", gpa: 3.2 },
];
describe("StudentTable", () => {
  it("shows loading spinner initially", async () => {
    renderWithProviders(<StudentTable />);
    expect(screen.getByText("Loading...")).toBeInTheDocument();
  });
  it("renders student rows after loading", async () => {
    const preloadedState = {
      studentApi: {
        queries: {
          "getStudents(undefined)": {
            status: "fulfilled",
            data: mockStudents,
          },
        },
      },
    };
    renderWithProviders(<StudentTable />, { preloadedState });

    await waitFor(() => {
      expect(screen.getByText("Alice")).toBeInTheDocument();
      expect(screen.getByText("3.80")).toBeInTheDocument();
    });
  });
});
```

## RTL API ที่ใช้

### `renderWithProviders()`

เรียกแทน `render()` ทุกครั้งที่ต้องการ Redux

### `screen.getByRole()`

หา element ด้วย ARIA role (progressbar, table ฯลฯ)

### `screen.getByText()`

หา element ที่มีข้อความตรงกับที่ระบุ

### `waitFor()`

รอ async เช่น เมื่อ API โหลดเสร็จแล้ว assert

### `preloadedState`

ตั้งค่า store ล่วงหน้าเพื่อข้าม loading state



# RTL Query Methods & Test Utilities

เครื่องมือค้นหา element และจัดการ async ใน React Testing Library

## screen.getByRole()

หา element ด้วย ARIA role

```
getByRole('progressbar')
getByRole('table')
getByRole('button', { name: 'Save' })
```

ใกล้เคียงกับวิธีที่ผู้ใช้งานมองหา element จริง ๆ

## screen.getByText()

หา element ที่มีข้อความตรงกับที่ระบุ

```
getByText('Alice')
getByText(/loading/i) // regex
```

ใช้ regex เมื่อข้อความอาจมีหลายรูปแบบ

## waitFor()

รอ async — retry จนกว่า callback จะไม่ throw

```
await waitFor(() =>
  expect(screen.getByText('Alice'))
    .toBeInTheDocument()
)
```

ใช้เมื่อ API โหลดเสร็จแล้วค่อย assert

## preloadedState

ตั้งค่า store ล่วงหน้าเพื่อข้าม loading state

```
renderWithProviders(<C />, {
  preloadedState: {
    students: {
      status: 'succeeded',
      list: [{ name: 'Alice' }]
    }
  }
})
```

# ARIA คืออะไร?

## Accessible Rich Internet Applications

### ความหมาย

### Accessible Rich Internet Applications

W3C spec ที่กำหนด **role, state, property** พิเศษให้ HTML elements เพื่อให้ screen reader และ assistive technology เข้าใจ UI ได้

ตัวอย่าง: role attribute

```
// ไม่มี ARIA — screen reader 'ไม่รู้'ว่าคืออะไร
<div class="spinner"></div>
```

```
// มี ARIA — screen reader รู้ว่าคือ progress bar
<div role="progressbar"></div>
```

getByRole() ค้นหา element ผ่าน ARIA role — เหมือน screen reader มองเห็น

### getByRole() ใน RTL

'progressbar' — แถบ loading / spinner

'table' — ตารางข้อมูล

'button' — ปุ่มกด

'heading' — หัวข้อ (h1-h6)

'textbox' — ช่องกรอกข้อมูล

'alert' — ข้อความแจ้งเตือน

เพิ่ม option ชื่อ:

```
getByRole('button', { name: 'Save' })
```

ระบุชื่อปุ่มเพิ่มเติมเมื่อมีหลายปุ่มในหน้าเดียวกัน

# การทดสอบ Selectors (createSelector)

Selectors เป็น pure function — ทดสอบโดยส่ง state object โดยตรง

## studentsSelectors.test.js

```
import { selectTopStudents } from "../selectors";
import { describe, it, expect } from "vitest";

describe("selectTopStudents", () => {
  const state = {
    studentApi: {
      queries: {
        "getStudents(undefined)": {
          data: [
            { id: 1, name: "Alice", studentId: "S001", major: "CS", gpa: 3.8 },
            { id: 2, name: "Bob", studentId: "S002", major: "Math", gpa: 3.2 },
            { id: 3, name: "Carol", studentId: "S003", major: "IT", gpa: 3.7 },
          ],
        },
      },
    },
  };

  it("returns students with gpa >= 3.5", () => {
    const result = selectTopStudents(state);
    expect(result).toHaveLength(2);
    expect(result[0].name).toBe("Alice");
    expect(result[1].name).toBe("Carol");
  });

  it("returns [] when status is not succeeded", () => {
    const s = { studentApi: { queries: {} } };
    expect(selectTopStudents(s)).toEqual([]);
  });
});
```

## หลักการและ API

### createSelector()

สร้าง memoized selector — คำนวณใหม่เฉพาะเมื่อ input เปลี่ยน

### ส่ง state โดยตรง

ไม่ต้องใช้ store — selector รับ state object ธรรมดา

### toHaveLength(n)

ตรวจสอบจำนวน element ใน array ที่คืนจาก selector

### toEqual([])

ตรวจสอบ array ว่าว่างเปล่าแบบ deep equal

### กรณีทดสอบขอบ

ทดสอบ status ที่ไม่ใช่ succeeded เพื่อตรวจสอบ edge case

## SESSION 8

---

# MSW — การจำลอง API

Mock Service Worker สำหรับการทดสอบ Network Requests

# MSW (Mock Service Worker) คืออะไร?

MSW ดักจับ HTTP requests ระดับ network ก่อนที่จะออกจาก browser

## หลักการทำงาน

- สร้าง Service Worker ที่ดักจับ fetch/XHR requests
- Handler function กำหนดว่าจะตอบสนองอย่างไร
- fetchBaseQuery ของ RTK Query ได้รับ response เหมือน API จริง
- ทำงานได้ใน browser และ Node.js (สำหรับ Vitest)
- ไม่ต้องแก้ source code เพื่อ mock — interceptor อยู่นอก app

## ทำไมถึงใช้ MSW?

- Test จริงผ่าน network layer — ครอบคลุมกว่าการ mock module
- Handlers ใช้ร่วมกันได้ระหว่าง dev, test, และ Storybook
- Override handler ใน test เฉพาะเพื่อทดสอบ error scenarios
- ไม่มี network call จริง — ทดสอบเร็ว ไม่ขึ้นกับ server

ติดตั้ง: `npm install msw --save-dev`

# MSW Handlers — กำหนด Mock Responses

Handlers สะท้อน API endpoints จริงของโปรเจกต์

## src/tests/handlers/students.js

```
import { http, HttpResponse } from 'msw'

export const studentHandlers = [
  http.get('/api/students', () => {
    return HttpResponse.json([
      { id: 1, name: 'Alice', gpa: 3.8 },
      { id: 2, name: 'Bob', gpa: 3.2 },
    ])
  }),
  http.post('/api/students', async ({ request }) => {
    const body = await request.json()
    return HttpResponse.json(
      { id: Date.now(), ...body }, { status: 201 }
    )
  }),
]
```

🔑 จุดสำคัญ

### http.get / http.post

จับ HTTP method + URL ที่ตรงกัน

### HttpResponse.json()

สร้าง JSON response พร้อม Content-Type

### status: 201

กำหนด HTTP status code

### request.json()

อ่าน request body จาก POST

### export handlers

นำไปรวมใน server.js

Handlers ทำงานใน Node.js environment — ไม่ต้องติดตั้ง Service Worker สำหรับ Vitest

# MSW Server & ไฟล์ตั้งค่าการทดสอบ

สร้าง server และเชื่อมต่อกับ Vitest lifecycle hooks

## src/tests/server.js

```
import { setupServer } from 'msw/node'
import { studentHandlers } from
  './handlers/students'

export const server = setupServer(
  ...studentHandlers
)
```

## src/tests/setup.js

```
import { server } from './server'

beforeAll(() => server.listen())
afterEach(() => server.resetHandlers())
afterAll(() => server.close())
```

## Lifecycle Hooks

### afterEach — server.resetHandlers()

รีเซ็ต override handlers หลังแต่ละ test เพื่อป้องกัน state หลงเหลือ

### beforeAll — server.listen()

เริ่ม MSW server ก่อน test suite ทั้งหมด

### afterAll — server.close()

ปิด server หลัง test suite เสร็จสิ้น

### server.use(...handlers)

Override handler ขั้วคราวใน test เฉพาะ — รีเซ็ตโดย afterEach

### เพิ่มใน vite.config.js: test.setupFiles:

```
['./src/tests/setup.js']
```

resetHandlers() คือกุญแจสำคัญ — ป้องกัน test หนึ่งส่งผลต่อ test ถัดไป

# การทดสอบ Query ด้วย MSW

Dispatch query endpoint โดยตรงจาก store — MSW ส่ง response กลับผ่าน network

## studentsApi.test.js

```
import { renderWithProviders } from '../tests/utils'
import { studentsApi } from '../store/studentsApi'

test('fetchStudents returns data from MSW', async () => {
  const { store } = renderWithProviders(null)

  store.dispatch(
    studentsApi.endpoints.getStudents.initiate()
  )

  await waitFor(() => {
    const state = store.getState()
    const result = studentsApi.endpoints
      .getStudents.select() (state)
    expect(result.status).toBe('fulfilled')
    expect(result.data).toHaveLength(2)
  })
})
```

### ขั้นตอน

1. render component เปล่า เพื่อสร้าง store
2. dispatch initiate() เพื่อ trigger query
3. MSW handler ดักจับ request ส่ง mock data
4. waitFor() รอให้ query fulfilled
5. ตรวจสอบ status และ data ใน store

💡 waitFor() จาก @testing-library/react รอ async state updates อัตโนมัติ



## Step 1 — render component เปล่า เพื่อสร้าง store

การทดสอบ Query ด้วย MSW · ขั้นตอนที่ 1 / 5

### สิ่งที่เกิดขึ้น

เรียก `renderWithProviders(null)` เพื่อสร้าง Redux store ใหม่โดยไม่ต้องมี component จริง  
ฟังก์ชันนี้คืนค่า `{ store }` ซึ่งเราจะใช้ dispatch query ในขั้นตอนถัดไป

```
// render null เพื่อสร้าง store โดยไม่ render UI
const { store } = renderWithProviders(null)
```

### ทำไมต้อง render null?

เราต้องการทดสอบ query ใน `store` โดยตรง ไม่ใช่ผ่านการ component  
การ render `null` ทำให้ได้ store พร้อม RTK Query middleware โดยไม่ต้องมี UI  
เทียบเท่ากับการ setup store ด้วย `configureStore()` โดยตรง แต่สะดวกกว่าเพราะใช้ config เดิม



`renderWithProviders(null)` = สร้าง store เปล่าพร้อม MSW middleware รอรับ request

1

2

3

4

5

## Step 2 — dispatch initiate() เพื่อ trigger query

การทดสอบ Query ด้วย MSW · ขั้นตอนที่ 2 / 5

### สิ่งที่เกิดขึ้น

เรียก `.initiate()` บน endpoint เพื่อส่งให้ RTK Query ยิง HTTP request ไปยัง server  
การ dispatch นี้เหมือนกับที่ hook `useGetStudentsQuery()` ทำใน component จริง

```
// dispatch initiate() → ส่ง HTTP GET ไปยัง /api/students
store.dispatch(
  studentsApi.endpoints.getStudents.initiate()
)
```

### `.initiate()` คืออะไร?

เป็น action creator ที่ RTK Query สร้างให้อัตโนมัติทุก endpoint  
เมื่อ dispatch แล้ว RTK Query จะเช็ค cache ก่อน — ถ้ายังไม่มีข้อมูลจะยิง request ใหม่  
ใน test เราต้องการให้มันยิง request จริงเพื่อให้ **MSW** ดักจับได้

`.initiate()` = action creator ที่ trigger HTTP request เหมือน hook แต่เรียกได้จาก store โดยตรง

1

2

3

4

5

## Step 3 — MSW handler ดักจับ request ส่ง mock data

การทดสอบ Query ด้วย MSW · ขั้นตอนที่ 3 / 5

### MSW ทำงานอย่างไร?

**MSW (Mock Service Worker)** ดักจับ HTTP request ะดับ network ใน test environment จะใช้ **node HTTP interceptor** แทน Service Worker

เมื่อ RTK Query ยิง **GET /api/students**:

- 1 MSW handler ตรวจสอบ URL pattern
- 2 ถ้าตรงกับ **http.get('/api/students')** จะ intercept
- 3 ส่ง mock response กลับแทน real server
- 4 RTK Query รับ response เก็บใน store

### MSW handler (handlers.js)

```
import { http, HttpResponse } from 'msw'

export const handlers = [
  http.get('/api/students',
    () => {
      return HttpResponse.json([
        { id:'1', name:'Alice', gpa: 3.9 },
        { id:'2', name:'Bob', gpa: 3.5 },
      ])
    }
  ),
]
```

# Step 4 — waitFor() รอให้ query fulfilled

การทดสอบ Query ด้วย MSW · ขั้นตอนที่ 4 / 5

สิ่งที่เกิดขึ้น 1

**waitFor()** จาก RTL จะ retry callback ซ้ำ ๆ จนกว่าจะไม่ throw หรือ timeout 2

ใช้รอ async operation เช่น HTTP request ที่ MSW กำลัง process อยู่ 3

```
// รอจนกว่า query จะเสร็จ แล้วค่อย assert
await waitFor(() => {
  const state = store.getState()
  const result = studentsApi.endpoints.getStudents.select()(state)
})
```

 4 5

สถานะที่ waitFor() รอผ่าน:

|                                                   |                                                     |                                                   |
|---------------------------------------------------|-----------------------------------------------------|---------------------------------------------------|
| <b>pending</b><br>กำลัง fetch — query ยังไม่เสร็จ | <b>fulfilled</b><br>fetch สำเร็จ — มีข้อมูลใน store | <b>rejected</b><br>fetch ล้มเหลว — error ใน store |
|---------------------------------------------------|-----------------------------------------------------|---------------------------------------------------|

💡 **waitFor()** retry โดย default ทุก 50ms นาน 1000ms — ตั้ง timeout ได้ด้วย { timeout: 3000 }

## Step 5 — ตรวจสอบ status และ data ใน store

การทดสอบ Query ด้วย MSW · ขั้นตอนที่ 5 / 5

### code ที่ใช้ assert

```
const state = store.getState()
const result = studentsApi.endpoints
  .getStudents.select()(state)

expect(result.status).toBe('fulfilled')
expect(result.data).toHaveLength(2)
// result.data = [
//   { id:'1', name:'Alice', gpa:3.9 },
//   { id:'2', name:'Bob', gpa:3.5 }
// ]
```

### .select() คืออะไร?

selector factory ที่ RTK Query สร้างให้อัตโนมัติ

รับ state แล้วคืน cache entry ของ query นั้น

เทียบกับ `useGetStudentsQuery()` แต่ใช้ใน test ได้โดยไม่ต้อง render

### result object มีอะไรบ้าง?

**status:** 'fulfilled' | 'pending' | 'rejected'

สถานะล่าสุดของ query

**data:** Student[]

ข้อมูลที่ได้จาก MSW response

**error:** FetchBaseQueryError

error เมื่อ status = rejected

**isLoading:** boolean

true ระหว่าง fetch ครั้งแรก

**isFetching:** boolean

true ระหว่าง fetch ทุกครั้ง

**isSuccess:** boolean

true เมื่อ status = fulfilled

1

2

3

4

5

# การทดสอบ Mutation ด้วย MSW

ทดสอบ success path และ error path ด้วย `server.use()` override

## ✓ success path

```
test('addStudent succeeds', async () => {
  const { store } = renderWithProviders(null)
  const result = await store.dispatch(
    studentsApi.endpoints.addStudent
      .initiate({ name: 'Carol', grade: 'A' })
  )
  expect(result.data.name).toBe('Carol')
})
```

## ✗ error path — `server.use()`

```
server.use(
  http.post('/api/students', () =>
    new HttpResponse(null, { status: 500 })
  )
)
```



ทดสอบทั้ง success และ error path เสมอเพื่อ coverage ที่สมบูรณ์

## 🔄 Two-path testing

### Default handler

ตอบกลับ 201 Created — สำหรับ success path

### `server.use()` override

แทนที่ด้วย 500 Error — ทดสอบ error handling

### afterEach reset

`resetHandlers()` คืน default handler ให้ test ถัดไป

### `expect(result.error)`

ตรวจสอบ error property บน mutation result

## SESSION 8

---

# การทดสอบรูปแบบ Async

Optimistic Updates และ Error Rollback ใน RTK Query

# การทดสอบ Optimistic Updates

ตรวจสอบว่า cache ถูกปรับปรุงทันทีก่อน server ตอบสนอง

## optimisticUpdate.test.js

```
test('optimistic update shows immediately', async () => {
  const preloadedState = {
    studentsApi: {
      queries: { 'getStudents(undefined)': {
        status: 'fulfilled',
        data: [{ id:1, name:'Alice', grade:'A' }]
      }}
    }
  }
  const { store } = renderWithProviders(<StudentTable />,
    { preloadedState })

  store.dispatch(studentsApi.endpoints
    .updateStudent.initiate({ id:1, grade:'B' }))

  await waitFor(() => {
    const data = getStudentsFromState(store.getState())
    expect(data[0].grade).toBe('B') // optimistic!
  })
})
```

### ✓ สิ่งที่ต้องตรวจสอบ

→ Cache update ทันที

grade เปลี่ยนก่อน server ตอบ

→ UI re-render

component แสดงค่าใหม่ทันที

→ Server confirm

หลัง response — ค่าใน cache ยืนยัน

→ No flicker

ไม่มี loading state ระหว่าง optimistic



Optimistic update: RTK Query ปรับ cache ก่อน แล้ว rollback ถ้า server ล้มเหลว



# การทดสอบ Error Rollback

Override handler ให้คืน 409 — ตรวจสอบว่า cache กลับสู่ค่าเดิม

## errorRollback.test.js

```
test('rolls back cache on 409 conflict', async () => {
  server.use(
    http.patch('/api/students/:id', () =>
      new HttpResponse(null, { status: 409 })
    )
  )
  const { store } = renderWithProviders(
    <StudentTable />, { preloadedState }
  )

  await store.dispatch(
    studentsApi.endpoints.updateStudent
      .initiate({ id:1, grade:'F' })
  )

  await waitFor(() => {
    const data = getStudentsFromState(store.getState())
    expect(data[0].grade).toBe('A') // rolled back!
  })
})
```

## ⚠ Rollback Flow

1. Optimistic update: grade → 'F'
2. MSW returns 409 conflict
3. RTK Query ตรวจจ็ error
4. Cache rollback: grade กลับ 'A'
5. expect(grade).toBe('A') ✓

💡 RTK Query จัดการ rollback อัตโนมัติผ่าน onQueryStarted lifecycle hook

## SESSION 8

---

# Build & Deploy

Production Build, Code Splitting และ Deploy ไปยัง Vercel

# การตั้งค่า Vite Production Build

เพิ่มส่วน build ใน vite.config.js เพื่อควบคุม output และ code splitting

## vite.config.js – build section

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  build: {
    outDir: 'dist',
    sourcemap: false,          // ปิด source map ใน prod
    chunkSizeWarningLimit: 500,
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
          redux: ['@reduxjs/toolkit', 'react-redux'],
        }
      }
    }
  }
})
```

รัน: `npm run build` — ไฟล์ทั้งหมดจะอยู่ใน /dist พร้อม deploy

## ⚙️ ตัวเลือกสำคัญ

### outDir

โฟลเดอร์ output (default: dist)

### sourcemap

false ใน prod ลด file size

### manualChunks

แยก vendor/redux เป็น chunk ต่างหาก

### chunkSizeWarning

เตือนเมื่อ chunk ใหญ่เกิน 500kB

# Code Splitting ด้วย React.lazy()

โหลด page component เฉพาะ route ที่ผู้ใช้เข้าถึง — ลด bundle size

## Before — all pages loaded eagerly

```
// App.jsx — loads all pages upfront
{ StudentsPage } from './pages/StudentsPage'
{ ReportsPage }   from './pages/ReportsPage'
{ SettingsPage }  from './pages/SettingsPage'
```

## After — lazy loaded per route

```
const StudentsPage = lazy(
  () => loadPage('StudentsPage')
)
const ReportsPage = lazy(
  () => loadPage('ReportsPage')
)

// In JSX:
<Suspense fallback={<Loading />}>
  <Routes>
    <Route path="/" element={<StudentsPage/>} />
    <Route path="/reports" element={<ReportsPage/>} />
  </Routes>
</Suspense>
```

## ⚡ ประโยชน์

→ Initial bundle ลดลง

โหลดเฉพาะ JS ของ route ปัจจุบัน

→ Parallel chunk downloads

แต่ละ chunk โหลดพร้อมกันได้

→ Better caching

chunks แยกกัน — update หนึ่งไม่กระทบอีก

→ Suspense fallback

แสดง loading UI ระหว่างรอ chunk

💡 Vite แยก lazy() chunks จัดโน้ตเมื่อ build — ไม่ต้องตั้งค่าเพิ่ม

# Deploy AcadeMate ไปยัง Vercel

สองวิธีสำหรับ deploy — CLI หรือ Git Integration

## Method 1 — Vercel CLI

```
# Install Vercel CLI
npm install -g vercel

# Build and deploy
npm run build
vercel --prod

# Follow prompts to link project
# URL returned immediately
```

## vercel.json (SPA routing fix)

```
{
  "rewrites": [
    { "source": "/(.*)",
      "destination": "/" }
  ]
}
```

vercel.json ป้องกัน 404 เมื่อ refresh หน้า — สำคัญสำหรับ React Router

## Method 2 — Git Integration

1. Push โค้ดไปยัง GitHub repository
2. เข้า vercel.com → New Project
3. เลือก repository และ branch
4. ตั้ง Build Command: npm run build
5. ตั้ง Output Directory: dist
6. Deploy — auto-deploy ทุก push

# Environment Variables ใน Vite + Vercel

ต้องใช้ prefix VITE\_ — ถูก inject เวลา build ไม่ใช่ runtime

## `.env` / `.env.production`

```
# Only VITE_ prefix is exposed to browser
VITE_API_URL=https://api.academate.app
VITE_APP_TITLE=AcadeMate

# NO prefix — server only, NOT in bundle
SECRET_KEY=do-not-expose-this
```

## Usage in source code

```
// Access via import.meta.env
const apiUrl = import.meta.env.VITE_API_URL

// In studentsApi.js:
baseUrl: import.meta.env.VITE_API_URL
```

## ตั้งค่าบน Vercel

Project Settings → Environment Variables

เพิ่มแต่ละ VITE\_ variable ในแต่ละ environment

### Environments

Production, Preview, Development แยกกันได้  
Rebuild required

เปลี่ยน env var ต้องกด Redeploy เสมอ

### ห้ามใส่ secrets

ค่าถูก embed ใน JS bundle — ผู้ใช้เห็นได้

⚠ ห้ามเก็บ API keys หรือ secrets ใน VITE\_ variables — ถูก expose ใน browser bundle

## SESSION 8



# Lab 8

สร้างชุดทดสอบ AcadeMate และ Deploy สู่ Production

### Step 1

ทดสอบ StudentTable

3 cases: loading, success, error

### Step 2

ทดสอบ AddStudentForm

userEvent + form submission

### Step 3

Deploy ไปยัง Vercel

build + env vars + live URL

ไฟล์ใน src/tests/ • ผล: suite สีเขียวทั้งหมด + Vercel URL

# Lab 8 — ภาพรวมและไฟล์ที่ต้องสร้าง

สามแบบฝึกหัดเพื่อสร้างชุดทดสอบและ deploy AcadeMate

## ไฟล์ที่ต้องสร้าง / แก้ไข

```
src/
├── tests/
│   ├── utils.js      ← renderWithProviders
│   ├── setup.js      ← MSW lifecycle hooks
│   ├── server.js     ← setupServer(handlers)
│   └── handlers/
│       └── students.js ← http.get, http.post
├── components/
│   ├── StudentTable.test.js ← Lab Step 1
│   └── AddStudentForm.test.js ← Lab Step 2
├── vite.config.js     ← build + test config
├── vercel.json        ← SPA routing rewrite
└── .env.production   ← VITE_API_URL
```

## 📄 ขั้นตอน Lab

### Step 1 — StudentTable Tests

3 cases: loading spinner, render data, API error (500)

### Step 2 — AddStudentForm Tests

userEvent.type + click, ตรวจสอบ feedback ที่ UI

### Step 3 — Deploy to Vercel

npm run build → vercel --prod → ทดสอบ live URL

เป้าหมาย: test suite สี่เซียร์ทั้งหมด + AcadeMate live บน Vercel



# Lab Step 1 — ทดสอบ StudentTable

เขียน 3 test cases ครอบคลุมทุก state ของ component

## 📦 Imports & Setup

```
import { screen, waitFor } from '@testing-library/react'
import { renderWithProviders } from '../tests/utils'
import { http, HttpResponse } from 'msw'
import { server } from '../tests/server'
import StudentTable from './StudentTable'

const mockStudents = [
  { id:1, name:'Alice', gpa: 3.8 },
  { id:2, name:'Bob', gpa: 3.2 }
]
```

## ✅ Test 1: แสดง Loading...

```
it('shows loading state', async () => {
  renderWithProviders(<StudentTable />)
  expect(screen.getByText('Loading...')).toBeInTheDocument()
})
```

## ✅ Test 2: แสดงข้อมูลนักเรียน

```
it('renders student rows', async () => {
  renderWithProviders(<StudentTable />)
  await waitFor(() => {
    expect(screen.getByText('Alice')).toBeInTheDocument()
  })
})
```

## ✅ Test 3: Error 403

```
it('shows error on 403', async () => {
  server.use(
    http.get('/api/students', () =>
      HttpResponse.json(
        { error:'Forbidden' }, { status:403 }
      )
    )
  )
  renderWithProviders(<StudentTable />)
  await waitFor(() => {
    expect(screen.getByText(/error/i)).toBeInTheDocument()
  })
})
```

## สิ่งที่ต้องทดสอบ

✓ Loading state แสดงระหว่างดึงข้อมูล

✓ ข้อมูลนักเรียนแสดงหลัง fetch สำเร็จ

✓ Error message แสดงเมื่อ API คืน 403

⚠️ ใช้ `server.use()` เพื่อ override handler เฉพาะใน test นั้น — `resetHandlers()` คืนค่า default ให้อัตโนมัติ



MSW ดักจับ network จริง — ทดสอบ RTK Query เหมือน production

# Lab Step 2 — ทดสอบ AddStudentForm (1/2)

## Imports, Mock Setup และขั้นตอนที่ 1-2



### Imports & Mock Setup

```
import userEvent from '@testing-library/user-event'
import { screen, waitFor } from '@testing-library/react'
import { renderWithProviders } from '../tests/utils'
import { http, HttpResponse } from 'msw'
import { server } from '../tests/server'
import AddStudentForm from '../AddStudentForm'

it('adds a new student', async () => {
  // Step 1: setup user event
  const user = userEvent.setup()

  // Step 2: override POST handler
  server.use(
    http.post('/api/students', () =>
      HttpResponse.json({
        id: 3,
        name: 'Charlie',
        course: 'Testing'
      })
    )
  )

  renderWithProviders(<AddStudentForm />)
  // ... continued on next slide
})
```

### ขั้นตอนการทดสอบ (1-2)

#### Step 1: Setup userEvent

`userEvent.setup()` สร้าง instance  
ที่จำลองการพิมพ์และคลิกเหมือนผู้ใช้จริง  
ต้องเรียกก่อนทำ interaction ทุกครั้ง

#### Step 2: Override POST Handler

`server.use()` กำหนด mock response  
เฉพาะสำหรับ test นี้เท่านั้น  
Handler คืนข้อมูล Charlie เพื่อจำลอง POST สำเร็จ  
หลัง test จบ `resetHandlers()` คืนค่าเดิมอัตโนมัติ

▶ ต่อไป: Step 3 (Type & Click) และ Step 4 (waitFor)



`userEvent` ใช้ Pointer/Keyboard Events จริง — จำลองพฤติกรรมผู้ใช้ได้แม่นยำกว่า `fireEvent`

# Lab Step 2 — ทดสอบ AddStudentForm (2/2)

## User Interaction และการตรวจสอบผลลัพธ์

### User Interaction & Assertion

```
// (ต่อจากสไลด์ที่แล้ว)
// Step 3: type & click
await user.type(
  screen.getByLabelText('Name'),
  'Charlie'
)
await user.type(
  screen.getByLabelText('Course'),
  'Testing'
)
await user.click(
  screen.getByRole('button', { name: /add/i })
)

// Step 4: assert result
await waitFor(() => {
  expect(
    screen.getByText('Charlie')
  ).toBeInTheDocument()
})
})
```

### ขั้นตอนการทดสอบ (3-4)

#### Step 3: Type & Click

`user.type(el, text)` — พิมพ์ข้อความ  
ใช้ `getByLabelText` เพื่อหา input จาก label  
แต่ละตัวอักษรจะ trigger keyboard event จริง  
`user.click()` — คลิกปุ่ม Add  
ใช้ `getByRole('button')` เพื่อหาปุ่ม

#### Step 4: Assert with waitFor

`waitFor(callback)` — รอ async update  
RTK Query dispatch mutation → อัปเดต store  
component re-render แสดง 'Charlie' ใน list  
ถ้า Charlie ไม่ปรากฏ — test fail พร้อม error message



Test ครบ 4 ขั้นตอน: Setup → Mock → Interact → Assert



`waitFor` จำเป็นเพราะ RTK Query mutation เป็น async — DOM ยังไม่อัปเดตทันทีหลัง click

# ? ตรวจสอบความเข้าใจ

## โค้ดตั้งต้น

```
beforeAll(() => server.listen())
afterEach(() => server.resetHandlers())
afterAll(() => server.close())

it('test A — overrides to 403', async () => {
  server.use(http.get('/api/students', () => HttpResponse.json({}, {status:403})))
  // ... test runs with 403 ...
})
it('test B — calls useGetStudentsQuery', async () => { /* ??? */ })
```

คำถาม: ใน test B — useGetStudentsQuery จะได้รับ response อะไร?

- A ยังได้ 403 เพราะ server.use() เปลี่ยน handler แล้ว
- B ได้ 200 (default) เพราะ resetHandlers() คืน handler หลังแต่ละ test
- C เกิด Error เพราะไม่มี handler รองรับ
- D ได้ undefined เพราะ server ถูก reset แล้ว

✓ ตอบ B —



# สรุป AcadeMate — 6 Sessions

ครบ 6 Sessions — จาก Redux พื้นฐานสู่การทดสอบระดับ Production

## S1 — Redux Basics

Store, Action, Reducer, Provider, useSelector, useDispatch

## S3 — Async Thunk

Lifecycle pending/fulfilled/rejected, error handling, loading state

## S5 — Advanced Patterns

Optimistic updates, error rollback, code splitting, Vite deploy

## S2 — Redux Toolkit

createSlice, configureStore, createAsyncThunk พื้นฐาน

## S4 — RTK Query

createApi, fetchBaseQuery, auto-generated hooks, cache management

## S6 — Testing

Vitest + RTL + MSW — Unit, Integration, renderWithProviders